3.9.1、轮廓入门

[OpenCV：轮廓：入门]
https://docs.opencv.org/4.x/d4/d73/tutorial_py_contours_begin.html

1. 什么是 Contours（轮廓）？

轮廓 可以简单理解为：连接所有具有相同颜色或相同灰度值的连续边界点的曲线。

- 轮廓非常适合做 形状分析、物体检测 和 物体识别。
- 重要前提：为了得到更准确的轮廓，通常需要先把图像转为二值图像（黑白图像）。
  - 推荐方法：使用阈值处理（cv.threshold）或 Canny 边缘检测。
- 在 OpenCV 中，找轮廓就像在黑色背景上找白色物体。所以物体必须是白色，背景是黑色。

2. 如何找到轮廓？（核心函数：cv.findContours()）

先看一个完整的示例代码：

```
import numpy as np
import cv2 as cv

# 1. 读取图像
im = cv.imread('test.jpg')          # 替换成你自己的图片路径
assert im is not None, "文件读取失败，请检查路径"

# 2. 转为灰度图
imgray = cv.cvtColor(im, cv.COLOR_BGR2GRAY)

# 3. 二值化（阈值处理）
ret, thresh = cv.threshold(imgray, 127, 255, 0)

# 4. 查找轮廓
contours, hierarchy = cv.findContours(thresh, cv.RETR_TREE,
cv.CHAIN_APPROX_SIMPLE)

print("找到的轮廓数量:", len(contours))
```

关键解释：

- cv.findContours() 有 3 个主要参数：
  1. 第一个参数：二值图像（thresh）。

2. 第二个参数：轮廓检索模式（Retrieval Mode），常用 cv.RETR_TREE（树状结构，能表示轮廓的层级关系）或 cv.RETR_LIST。
3. 第三个参数：轮廓近似方法（Approximation Method）。

- 返回值：
  - contours：一个 Python 列表，列表里的每个元素是一个轮廓（NumPy 数组，形状为 (N, 1, 2)，存放边界点的 (x, y) 坐标）。
  - hierarchy：轮廓的层级信息（后面教程会详细讲）。

3. 如何绘制轮廓？（cv.drawContours()）

找到轮廓后，我们通常要把它们画出来看效果。

```
# 在原图上绘制所有轮廓（绿色，线宽 3）
cv.drawContours(im, contours, -1, (0, 255, 0), 3)

# 只绘制第 4 个轮廓（索引从 0 开始）
cv.drawContours(im, contours, 3, (0, 255, 0), 3)

# 更推荐的写法（单独处理某个轮廓）：
cnt = contours[4]                    # 取出第 5 个轮廓
cv.drawContours(im, [cnt], 0, (0, 255, 0), 3)
```

参数说明：

- im：要在哪张图上画（通常是彩色原图）。
- contours：轮廓列表。
- contourIdx：画第几个轮廓，-1 表示画全部。
- (0, 255, 0)：BGR 颜色，这里是绿色。
- 3：线条粗细。

4. 轮廓近似方法（Contour Approximation Method）

这是 findContours() 的第三个参数，非常重要！

方法	含义	例子（矩形）	优点
cv.CHAIN_APPROX_NONE	存储所有边界点	可能存 734 个点	最精确，但占用内存多
cv.CHAIN_APPROX_SIMPLE	压缩轮廓，只保留关键点（如拐角）	只存 4 个角点	节省内存，推荐使用

5. 完整实用代码示例

```
import cv2 as cv

img = cv.imread('your_image.jpg')
gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)

# 二值化（可以尝试不同阈值，或用 Otsu 方法）
ret, thresh = cv.threshold(gray, 127, 255, cv.THRESH_BINARY)

# 查找轮廓
contours, hierarchy = cv.findContours(thresh, cv.RETR_TREE,
cv.CHAIN_APPROX_SIMPLE)

# 绘制所有轮廓
cv.drawContours(img, contours, -1, (0, 255, 0), 2)

cv.imshow('Contours', img)
cv.waitKey(0)
cv.destroyAllWindows()
```

技巧：

- 如果背景很复杂，可以先用 cv.Canny() 做边缘检测，再找轮廓，效果往往更好。
- 阈值选择很重要：可以用 cv.THRESH_OTSU 自动选择阈值。